

C Program Implementation

Function Declaration and Summation

This example demonstrates the standard structure of a C program where a function is declared before use and defined after the main routine.

```
#include <stdio.h>

/* Function declaration */
int add(int x, int y);

int main() {
    int result = add(5, 3);    // function call
    printf("Result: %d", result); // print result
    return 0;
}

/* Function definition */
int add(int x, int y) {
    return x + y;
}
```

Brief Explanation

Step-by-Step Breakdown

- 1. Declaration:** The `add()` function is declared above `main()`. This tells the compiler that a function named 'add' exists and accepts two integers.
- 2. Execution:** Inside `main()`, the program calls `add(5, 3)`. The execution jumps to the function logic, passing 5 and 3 as arguments.
- 3. Storage:** The returned value (8) is stored in the integer variable `result`.
- 4. Output:** The `printf` function uses the `%d` format specifier to display the final sum on the console.
- 5. Definition:** The actual logic `return x + y;` is defined below `main()`, completing the implementation.

Conclusion

Key Takeaways for Codehive

Structuring functions this way is a best practice in C for several reasons:

- **Clarity:** High-level logic (the main function) remains at the top of the file for quick reading.
- **Compiler Compliance:** Prevents 'implicit declaration' warnings by providing a prototype early.
- **Documentation:** The function prototype acts as an immediate summary of inputs and outputs for the developer.

Final Result of Example Code: **8**